CS2040C+IT5003 Semester 2 2025/2026
Data Structures and Algorithms

Tutorial 10
SSSP + Wrap-up
For Week 13 (both courses)

Document is last modified on: April 10, 2026

1 Introduction and Objective

In this last tutorial, we will discuss the last examinable topic for CS2040C+IT5003 this semester: Single-Source Shortest Paths (SSSP) problem and continue talking about the 'graph modeling' soft skill, i.e., the ability to model a seemingly random (non-explicit-graph) problem into a graph problem (specifically the SSSP problem for this tutorial).

We will use `https://visualgo.net/en/sssp` during our discussion in this tutorial.

SSSP problem is quite easily found in many real-life applications and it is the source of many interesting Computer Science problems, as you can see in this tutorial. Again, we recommend that you put some thoughts on them before discussing the potential solutions with your tutor.

Update for CS2040C: Prof Halim has decided that since this semester we are offset by one week due to not having Week 06 classes, discussion on `https://visualgo.net/en/mst` is only done on Lecture 13a, there is no dedicated tutorial on MST.

2 Tutorial Questions

Standard Stuffs

So far, you have been presented with these SSSP algorithms: Bellman-Ford algorithm (without the proofs and only in a very high level manner - both courses), BFS (only for unweighted graph - both courses), Dijkstra's algorithm (the modification - so that we can use built-in min PQ data structure - both courses; (CS2040C also learns the original form of Dijkstra's, but IT5003 only learns the modification of Dijkstra's), and again, without the proofs). Additionally, CS2040C sides learn more about DFS/BFS (SSSP on Tree) and Topological Sort/DP (SSSP on DAG), whereas IT5003 side will likely just use Dijkstra's for all those variants.

First, the tutor will (re-)demonstrate the executions of Bellman-Ford, BFS, Modified Dijkstra's (both courses) a small directed weighted graph using `https://visualgo.net/en/sssp` from a certain source vertex s. CS2040C side will also show DFS, and/or Topological Sort/DP algorithms. The tutor will re-explain when a certain algorithm can be used and when the same algorithm cannot be used. The tutor may invite some students to do this live demonstration using different source vertex s and/or using different graph. If the class has no issue with the basics, tutor will proceed to discuss harder topics.

Graph Modeling Exercises

Q1). Let's discuss /minimum-genetic-mutation first that will later be used for lab demo, we need to realize that this is an unweighted SSSP problem on an implicit graph. TA will train students to recognize the tell-tale signs, i.e., what are the vertices represent?, what are the edges represent? directed or not? weighted or not?, what is the graph problem that we are trying to solve? what algorithm to use?

Next, there are a few graph questions in recent final assessment papers. Let's discuss three of them: (you can bet that there will be SSSP question(s) in the Final Assessment).

Q2). See Figure 1 and 2), Question 4.1, Facebook Privacy Setting
(shown at the bottom of this document).

Q3). Read `https://open.kattis.com/problems/emergency` and solve it!

Q4). See Figure 3, Question C.3, Avengers Initiative
(also shown at the bottom of this document).

Hands-on

TA will run the second half of this session with a few to do list:

- CS2040C B01/B04 only: Speedrun /graphds, /dfsbfs, and /sssp for last minute VisuAlgo Online Quiz 3 preparation:
  `https://visualgo.net/training?diff=Medium&n=5&tl=5&module=graphds,dfsbfs,sssp`
- Hands-on: Fully solve /minimum-genetic-mutation as discussed earlier above.

  Believe me (Prof Halim), /sssp problems are mostly on Medium/Hard level (not Easy) in LeetCode (as it uses understanding of PQ/bBST (dynamic sorting), DAT (visited flag, shortest distance updates), graph ds (EL, AL), etc). Between BFS (unweighted SSSP) versus Dijkstra's (weighted SSSP), Prof Halim reckons the ratio is approximately 2:1, information correct after solving 2034+/3893 (52%) of LeetCode tasks as of today)

Problem Set 6

We will end the tutorial with **high-level** discussion of PS6.
We can now discuss more ideas of PS6.
Final warning: Graph problem(s) will definitely appear in the final assessment.
Students need to be able to derive as much of these solutions themselves within 2 hours time constraint.

Remarks

This is the last tutorial for both CS2040C/IT5003 this semester.
TAs are encouraged to take class photo for momento and share the photo in Class Discord.

4.1 Facebook Privacy Setting (19 marks)

4.1.1 Definition

In Facebook, we can set our privacy setting so that only our Friends of Friends (and our direct friends) can look at our profile (see Figure 2). Our direct friends are classified as having degree 1 (one hop) to us. Our 'friends of friends' are therefore classified as having degree 2 (two hops away).

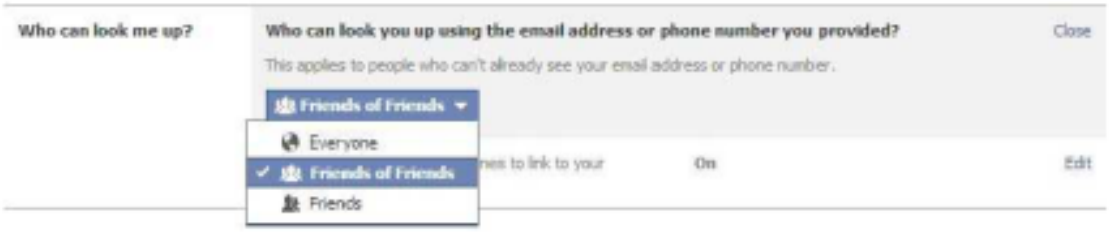

Figure 2: Restricting Access to Friends of Friends (degree 2) Only

Let's **assume** that we have **somehow** managed to store Facebook graph in an **Adjacency List** called `AdjList` (PS: We know that Facebook uses a much better graph data structure than this). Each user profile is given a unique integer index i. The friend list of profile i is stored in `AdjList[i]` and **sorted in ascending order**.

Now, we now want to check whether a user i can access user j's profile if user j sets his/her privacy setting to be degree 2 only as shown in Figure 2. The function `CanAccess(i, j)` should return true if i is classified as degree 2, 1, or 0 of user j, or return false otherwise[1].

4.1.2 (Optional) $O(V + E)$ Naive Solution (7 marks); Marks = ____

Please implement the function `CanAccess(i, j)` using a simple $O(V + E)$ naive solution.
Note that if you can already find the better $O(k)$ solution in Section 4.1.3, you can choose to leave this part blank and still get the full marks. But please write the $O(V + E)$ naive solution in case you have no clue or unsure with your $O(k)$ solution.

```
function Boolean CanAccess(int i, int j) { // write an O(V + E) solution

}
```

[1]This problem is from a real Google interview question, but it has been rewritten using another background story.

Figure 1: Facebook 1

4.1.3 $O(k)$ Efficient Solution (12 marks); Marks = ____ + ____ = ____

The size of V or E in a Facebook graph can be very big. Therefore, an $O(V+E)$ solution may not be the best solution. Please implement the function `CanAccess(i, j)` using an **efficient $O(k)$ solution** where $k = k_i + k_j$ and k_i/k_j is the number of friends of user i/j. The memory is also at premium and therefore we **cannot use additional data structure** other than a few extra variables. The grading scheme for this part is very strict, i.e. 0 (blank, incorrect, slower than $O(k)$, or uses additional data structure), or 10 (minor error(s)), 12 (fully correct).

```
function Boolean CanAccess(int i, int j) { // write an O(k) solution



}
```

Figure 2: Facebook 2

C.3 Avengers Initiative (30 marks)

There are A Avengers ($1 < A \leq 10\,000$, yes, a lot of them now) currently staying in various places (vertices) in the world. Suddenly, there is a major worldwide threat too great for the forces of mankind to handle that requires all of these A Avengers to assemble in one of H S.H.I.E.L.D. Helicarriers ($1 < H \leq 10$, yes, they have more than one now) currently stationed in various places (vertices) in the world. None of the Avenger is currently at any of the Helicarrier at this point of time. So, Nick Fury reactivates the 'Avengers Initiative' and call all these A Avengers to assemble.

Given a world map G that describes up to V places (vertices) in the world ($A + H \leq V \leq 100\,000$) and E weighted edges that describe the fastest direct connection between places in hours ($0 \leq E \leq 200\,000$, edge weight is a positive integer not more than 1000), determine which of the H Helicarriers should be chosen as the meeting point by Nick Fury and compute the shortest possible time (in hours) from Nick Fury's 'Avenger Initiative' reactivation before **all** these A Avengers can meet together and discuss the solution for that worldwide threat. If there is no Helicarrier where all A Avengers can assemble, you have to report 'Doomsday' instead.

For example, you are given a world map G as shown in Figure 3. In this example, the best meeting point is the Helicarrier at vertex 0 and the meeting can begin earliest 6 hours from now as Avenger at vertex 5 is the slowest to arrive (he/she needs 5+1 = 6 hours to traverse path $5 \rightarrow 1 \rightarrow 0$). The other 3 Avengers will be able to reach Helicarrier at vertex 0 in ≤ 6 hours. Note that if Nick Fury chooses Helicarrier 1 as the meeting point, then Avenger at vertex 4 will cause the bottleneck as he/she needs 4+9 = 13 hours to traverse path $4 \rightarrow 0 \rightarrow 1$. The world probably have ended by then.

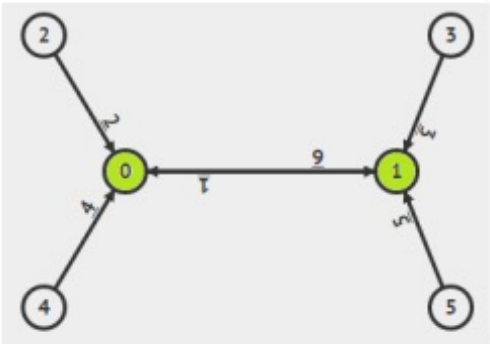

Figure 3: A Sample World Map G. There are $H = 2$ Helicarriers currently located at vertex $\{0, 1\}$. There are $A = 4$ Avengers currently located at vertex $\{2, 3, 4, 5\}$. For simplicity, no other intermediate vertices are shown in this Sample World Map. Storyline credit: Marvel Comics.

Design the best algorithm that you have learned in CS2010 (or beyond) to solve this problem and analyze the time complexity of your algorithm. The rough marking scheme is as follows:

- Not blank, but your algorithm is totally incorrect: 1 mark
- Your algorithm is incorrect: 5 marks
- Your algorithm is correct, but (much) slower than the examiner's best answer: 15 marks
- Your algorithm is correct and equal to/better than the examiner's best answer: 30 marks
- For correct algorithm, we deduct 5 marks for wrong time complexity analysis

Figure 3: Avengers Initiative